SBT-DF205 — Mobile and IoT Forensics Case Study

Case Study: Android Forensics Investigation of a Google Pixel 3

Student name: Afeez Bolaji Usman

Student ID: 202514326WY

Course: SBT-DF205-Mobile and IoT Forensics Case Study

Lab Title:

Instructor Name: Aminu Idris, AMCPN
Date: 27/10/2026

Executive Summary

This investigation examines the supplied Android file-system extraction from a Google Pixel 3 to identify and assess evidence relevant to suspected unauthorised communication and information sharing at Harbour Operations Ltd. The examination covers device identification and user profiles, system settings and connectivity, SMS and MMS messages, contacts and call logs, Google Messages and Dialer, WhatsApp and another messaging application, Google Maps records, and photographic metadata. Forensic methods include file integrity verification, SQLite database examination, metadata analysis, cross-referencing records from different sources, and timeline reconstruction. Original record identifiers, source paths, and raw timestamps are retained to support the accuracy, traceability, and reproducibility of the findings. The allegation will be assessed using verified evidence from the supplied extraction. The final conclusion will distinguish corroborating findings from contradictory or inconclusive evidence, consider alternative explanations, and acknowledge any limitations affecting the attribution of activity to a particular user.

Objectives

1. **Evidence preservation and integrity:** Preserve the supplied evidence, verify its integrity using SHA-256 hashes, and maintain an evidence register and handle log documenting all examination activities.

2. **Device identification and user attribution:** Establish the device manufacturer, model, Android version, build identifier, security patch level, and available user profiles or account associations.

3. **System settings and connectivity:** Examine available secure, global, and system settings, together with Wi-Fi and Bluetooth configuration records, to identify relevant device settings and connectivity information.

4. **Communication and personal records:** Examine SMS, MMS, contacts, call logs, and calendar records; reconstruct relevant communications; and compare available database copies to identify consistency, discrepancies, or missing records.

5. **Cross-source communication verification:** Compare Google Messages records with the Android Telephony Provider and Google Dialer records with the AOSP call log to identify matching records and assess their evidential significance.

6. **Third-party messaging applications:** Examine WhatsApp and either TextNow or Kik for available account associations and communication records, and correlate relevant findings with other artefacts where possible.

7. **Location and photographic metadata:** Examine Google Maps location-related records and photographic metadata, documenting available timestamps, time-zone offsets, file hashes, and GPS information.

8. **Timeline reconstruction and allegation assessment:** Construct a timeline containing at least 12 distinct events from four or more artefact families, identify at least three cross-source correlations, and assess whether the allegation of unauthorised communication and information sharing is supported, contradicted, or unresolved based on the verified evidence.

The investigation will document source paths, record identifiers, relevant limitations, and alternative interpretations to ensure that conclusions remain traceable to the available evidence.

Tools Used

Tool	Purpose

Kali Linux	Forensic working environment
7-Zip (7z)	Archive integrity testing and extraction
SHA-256 (sha256sum)	Evidence integrity verification
stat and file	File metadata and file-type identification
SQLite (sqlite3)	Database schema examination, record extraction and SQL queries
ExifTool	Photographic metadata examination
Text editor	Inspection of XML, configuration files and supporting records
Spreadsheet editor	Review and preparation of evidence register and timeline CSV files
Screenshot utility	Capture of examination commands and supporting results

```
┌──(kali㉿kali)-[~/Pixel3]
└─$ wget -q https://digitalcorpora.s3.amazonaws.com/corpora/mobile/android_10/Non-Cellebrite%20Extraction/Pixel%203.zip


┌──(kali㉿kali)-[~/Pixel3]
└─$ ls Pixel\ 3.zip -l
-rw-rw-r-- 1 kali kali 3753183224 Oct 26 21:00 'Pixel 3.zip'

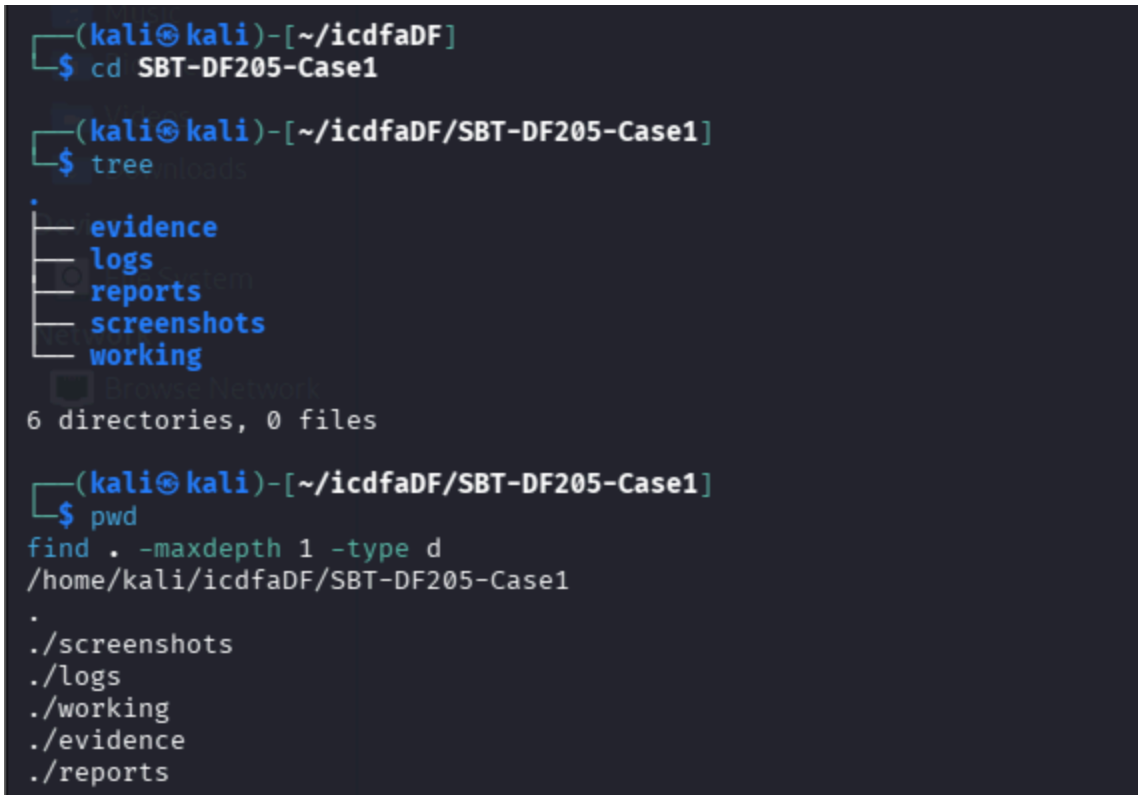
┌──(kali㉿kali)-[~/Pixel3]
└─$ hashdeep Pixel\ 3.zip
%%%% HASHDEEP-1.0
%%%% size,md5,sha256,filename
## Invoked from: /home/kali/Pixel3
## $ hashdeep Pixel 3.zip
##
3753183224,c79a39a301f65c6365911fc63ed6fa4e,5ed39583b6adfb01b5b073d4c4e85dae7dec2c94f9d547d77b8ed769e8cd7978,/home/kali/Pixel3/Pixel 3.zip
```

Commands

Open Kali Terminal and run:

cd ~/icdfaDF/SBT-DF205-Case1

mkdir -p evidence working/extracted reports screenshots logs



Methodology

The examination follows a preservation-first, file-system-based digital forensic workflow designed to maintain evidence integrity, support reproducibility, and ensure that conclusions are grounded in verified records.

- **Evidence preservation and integrity:** Identify the supplied archive and document its source, filename, file size, and receipt time. Calculate its SHA-256 hash before extraction, test the archive's integrity, and recalculate the original archive's hash after processing to check for changes.

- **Working environment:** Preserve the original archive and conduct extraction and analysis in a separate working directory. Retain SQLite database write-ahead log (`-wal`) and shared-memory (`-shm`) sidecar files alongside their corresponding databases to avoid overlooking relevant records.

- **Device identification:** Examine relevant `build.prop` files and Android user and profile records. Document the source paths and relevant stored property keys, values, and identifiers.

- **Database examination:** Identify available SQLite databases, inspect their schemas, record table names and row counts, and execute documented SQL queries to extract relevant communications, contacts, calls, calendar entries, and application records.

- **Application analysis:** Examine Google Messages, Google Dialer, WhatsApp, and an available alternative messaging application. Compare related records across data sources and distinguish duplicate representations of the same event from separate events.

- **Settings and media examination:** Analyse available system settings, Wi-Fi and Bluetooth configurations, Google Maps records, and photographic metadata. Record relevant file hashes and distinguish stored configurations or metadata from evidence demonstrating that an activity actually occurred.

- **Timeline reconstruction:** Preserve raw timestamp values and formats, verify timestamp units and applicable time-zone assumptions, and convert timestamps to Coordinated Universal Time (UTC) only when supported by the available evidence.

- **Reporting and verification:** Maintain an evidence register, handling log, documented SQL queries, screenshot collection, and event timeline. Evaluate the allegation using verified findings, consider alternative explanations, and document evidence gaps, technical limitations, and uncertainties affecting attribution.

This methodology supports a transparent and reproducible examination while ensuring that findings and conclusions do not extend beyond what the available evidence can establish.

Findings — Task 1: Evidence Preservation

The supplied evidence archive was identified as Pixel 3.zip, located at evidence/Pixel 3.zip within the case directory. The archive was identified as a ZIP64 archive with a recorded size of 5,247,820,897 bytes.

Before extraction, the archive's SHA-256 hash was calculated as:

ca6918ef8b20486b6a5ded15609ac51318f377829480f93be3ba15364a8aa00a

The archive was tested using 7-Zip version 26.03. The test completed with the result Everything is Ok. The archive was then extracted into the working/extracted directory. The extraction output reported 64,550 files, 11,853 folders and a total extracted size of 8,991,963,938 bytes.

Following extraction, the original archive was rehashed using sha256sum -c logs/archive_hash_initial.txt. The verification returned evidence/Pixel 3.zip: OK, confirming that the archive matched its initial SHA-256 baseline at the time of rechecking.

The initial hash is retained as the examination baseline. No trusted reference hash or manifest comparison has yet been documented. The successful integrity checks establish consistency with the recorded baseline, not independent proof of the archive's authenticity before receipt.

Commands

cd ~/icdfaDF/SBT-DF205-Case1

ls -lh evidence/



Calculate the initial SHA-256

Command

sha256sum evidence/Pixel\ 3.zip \

 | tee logs/archive_hash_initial.txt



Sha256sum hash :
ca6918ef8b20486b6a5ded15609ac51318f377829480f93be3ba15364a8aa00a evidence/Pixel 3.zip

Test the archive

Command

stat "evidence/ACTUAL_ARCHIVE_NAME" \

 | tee logs/archive_stat.txt

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ stat evidence/Pixel\ 3.zip \
  | tee logs/archive_stat.txt
  File: evidence/Pixel 3.zip
  Size: 5247820897      Blocks: 10249664   IO Block: 4096   regular file
Device: 8,1     Inode: 837838      Links: 1
Access: (0766/-rwxrw-rw-)  Uid: ( 1000/    kali)   Gid: ( 1000/    kali)
Access: 2026-10-27 08:02:25.702130243 -0400
Modify: 2026-10-26 19:50:15.389304000 -0400
Change: 2026-10-27 07:59:08.942021519 -0400
 Birth: 2026-10-27 07:33:49.917091848 -0400
```

file "evidence/ACTUAL_ARCHIVE_NAME"

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ file evidence/Pixel\ 3.zip
evidence/Pixel 3.zip: Zip archive data, at least v2.0 to extract, compression method=store

┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$
```

Test the archive:

7z t "evidence/ACTUAL_ARCHIVE_NAME" \

2>&1 | tee logs/archive_test.txt

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ 7z t "evidence/Pixel 3.zip" \
  2>&1 | tee logs/Pixel 3.txt

7-Zip 26.03 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-09-03
 64-bit locale=en_US.UTF-8 Threads:128 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 5247820897 bytes (5005 MiB)

Testing archive: evidence/Pixel 3.zip
--
Path = evidence/Pixel 3.zip
Type = zip
Physical Size = 5247820897
64-bit = +
Characteristics = Zip64

Everything is Ok

Folders: 11853
Files: 64550
Size:       8991963938
Compressed: 5247820897
```

Extract the archive

mkdir -p working/extracted

Command

7z x "evidence/ACTUAL_ARCHIVE_NAME" \

-oworking/extracted \

2>&1 | tee logs/extraction_log.txt

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ 7z x evidence/Pixel\ 3.zip \
  -oworking/extracted

7-Zip 26.03 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-09-03
 64-bit locale=en_US.UTF-8 Threads:128 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 5247820897 bytes (5005 MiB)

Extracting archive: evidence/Pixel 3.zip
--
Path = evidence/Pixel 3.zip
Type = zip
Physical Size = 5247820897
64-bit = +
Characteristics = Zip64

Everything is Ok

Folders: 11853
Files: 64550
Size:       8991963938
Compressed: 5247820897
```

Verify the original archive hash again

Command

sha256sum -c logs/archive_hash_initial.txt

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ sha256sum -c logs/archive_hash_initial.txt
evidence/Pixel 3.zip: OK
```

Findings — Device Identity and Android Build

The examination identified multiple build.prop files within the supplied extraction. The files working/extracted/Pixel 3/sbin/.magisk/mirror/system_root/system/build.prop and working/extracted/Pixel 3/system/build.prop contained the following relevant properties:

Property key	Observed value
ro.product.system.brand	google
ro.product.system.device	generic
ro.product.system.manufacturer	Google
ro.product.system.model	mainline
ro.build.id	QQ1A.200105.003
ro.build.display.id	QQ1A.200105.003
ro.build.version.release	10
ro.build.version.security_patch	2020-01-01

The examined system properties identify Google as the system manufacturer and brand, Android version 10, build identifier QQ1A.200105.003, and security patch level 2020-01-01.

The device and model properties returned generic and mainline, respectively. These values do not independently verify the physical handset model. The case brief identifies the supplied evidence as a Google Pixel 3; independent corroboration from additional device records is required before treating that model identification as established by the examined properties.

The same values appeared in two system property files. Because these files may represent duplicated or mirrored system content, they are not treated as independent corroboration.

Locate build.prop

Command

find working/extracted -type f -name 'build.prop' \

-print | tee logs/build_prop_paths.txt

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ cd ~/icdfaDF/SBT-DF205-Case1

find working/extracted -type f -name 'build.prop' \
  -print | tee logs/build_prop_paths.txt
working/extracted/Pixel 3/product/build.prop
working/extracted/Pixel 3/vendor/odm/etc/build.prop
working/extracted/Pixel 3/vendor/build.prop
working/extracted/Pixel 3/sbin/.magisk/mirror/system_root/system/build.prop
working/extracted/Pixel 3/sbin/.magisk/mirror/product/build.prop
working/extracted/Pixel 3/sbin/.magisk/mirror/vendor/odm/etc/build.prop
working/extracted/Pixel 3/sbin/.magisk/mirror/vendor/build.prop
working/extracted/Pixel 3/system/build.prop

┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
```

Extract the required device properties

Command

while IFS= read -r f; do

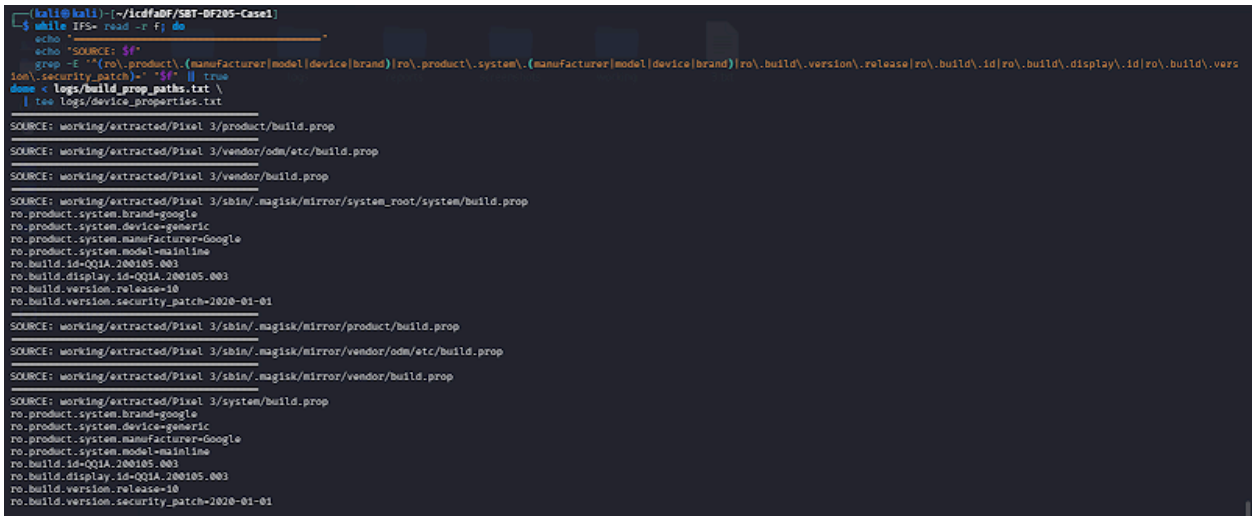
echo "==="

echo "SOURCE: $f"

grep -E '^(ro\.product\.(manufacturer|model|device|brand)|ro\.product\.system\.(manufacturer|model|device|brand)|ro\.build\.version\.release|ro\.build\.id|ro\.build\.display\.id|ro\.build\.version\.security_patch)=' "$f" || true

done < logs/build_prop_paths.txt \

  | tee logs/device_properties.txt



Findings — Android User Profiles

The extracted Android user registry, data/system/users/userlist.xml, contains two user entries: ID 0 and ID 11. The corresponding profile files are data/system/users/0.xml and data/system/users/11.xml.

Profile ID 0 has the displayed name “This Is” and records a lastLoggedIn value of 1581695517124 milliseconds. Profile ID 11 has the displayed name “User 2”, a creation value of 1581644358241 milliseconds, and a lastLoggedIn value of 1581684601922 milliseconds. Both profile records contain the build fingerprint google/blueline/blueline:10/QQ1A.200105.003/6079943:user/release-keys.

These records support the existence of two Android user profiles in the supplied extraction. The displayed profile names are not independently verified identities. Profile login fields are recorded system metadata and should not, without corroboration, be treated as proof of who physically operated the device.

Locate profile files

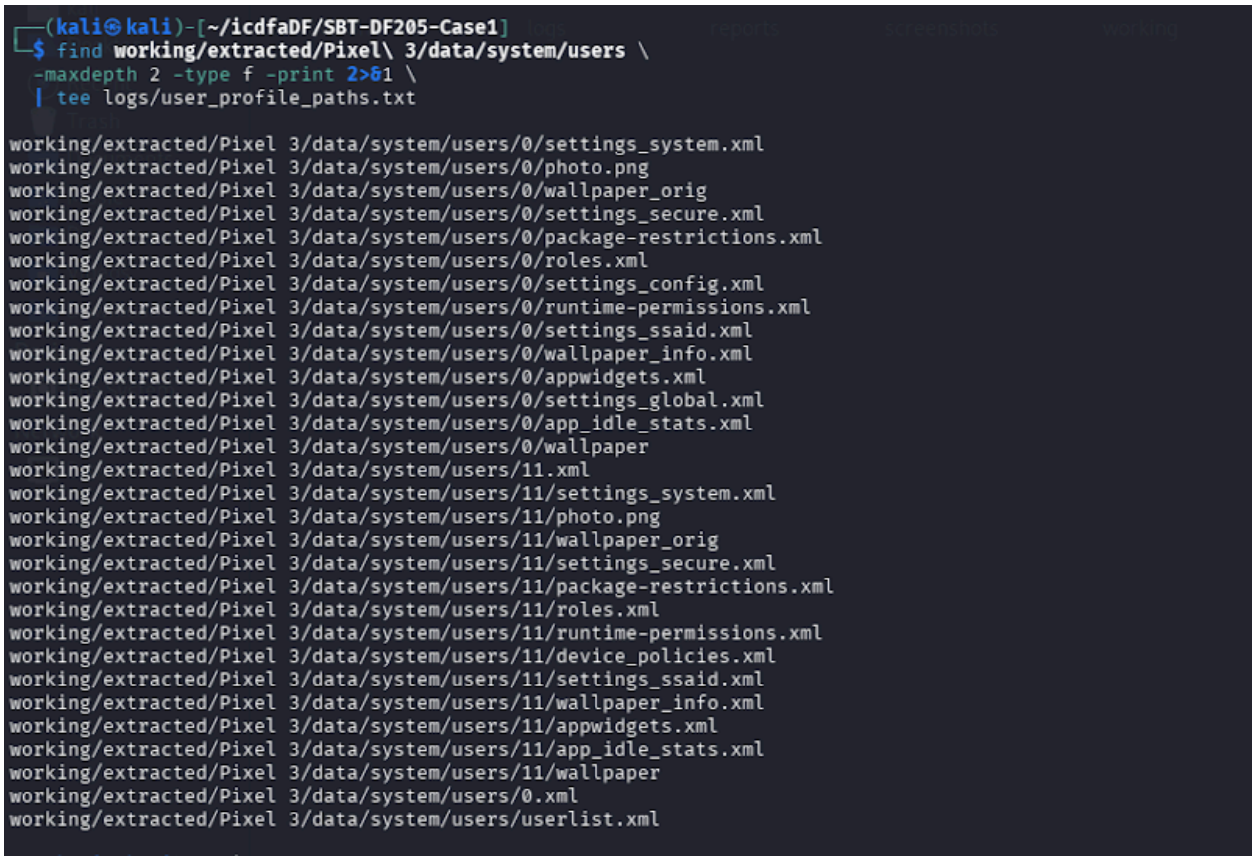
Run:

cd ~/icdfaDF/SBT-DF205-Case1

find working/extracted/Pixel\ 3/data/system/users \

  -maxdepth 2 -type f -print 2>&1 \

  | tee logs/user_profile_paths.txt



Inspect profile XML files

while IFS= read -r f; do

  case "$f" in

```
        *.xml)
            echo "========================================="
            echo "SOURCE: $(sed -n '1,240p' "$f")"
            ;;
    esac
done < logs/user_profile_paths.txt \
  | tee logs/user_profile_details.txt
```



Account Associations and Corroboration

The Android account database for user profile 0 contains the Google account thisisdfir@gmail.com with account type com.google. The same identifier appears in Gmail's warm_accounts_preferences.xml and Google Play's lastAccount.xml, where it is stored under the preference key lastAccountStr.

The device-protected account database for user profile 11 contains thisisdfirtwo@gmail.com with account type com.google. This is evidence of a Google account record associated with that profile, although further application-level corroboration for this second account has not yet been established.

The records support the presence of these account identifiers in the extracted data. They do not independently establish the real-world identity of the account holder or prove who operated the device.

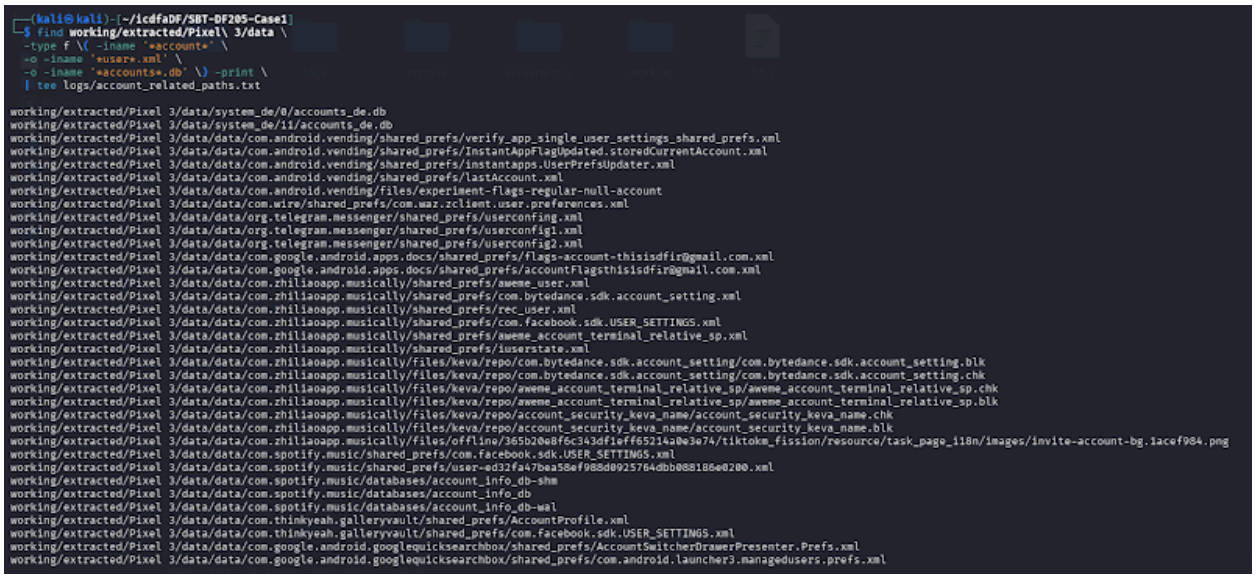
find working/extracted/Pixel\ 3/data \

-type f \(-iname '*account*' \

-o -iname '*user*.xml' \

-o -iname '*accounts*.db' \) -print \

| tee logs/account_related_paths.txt

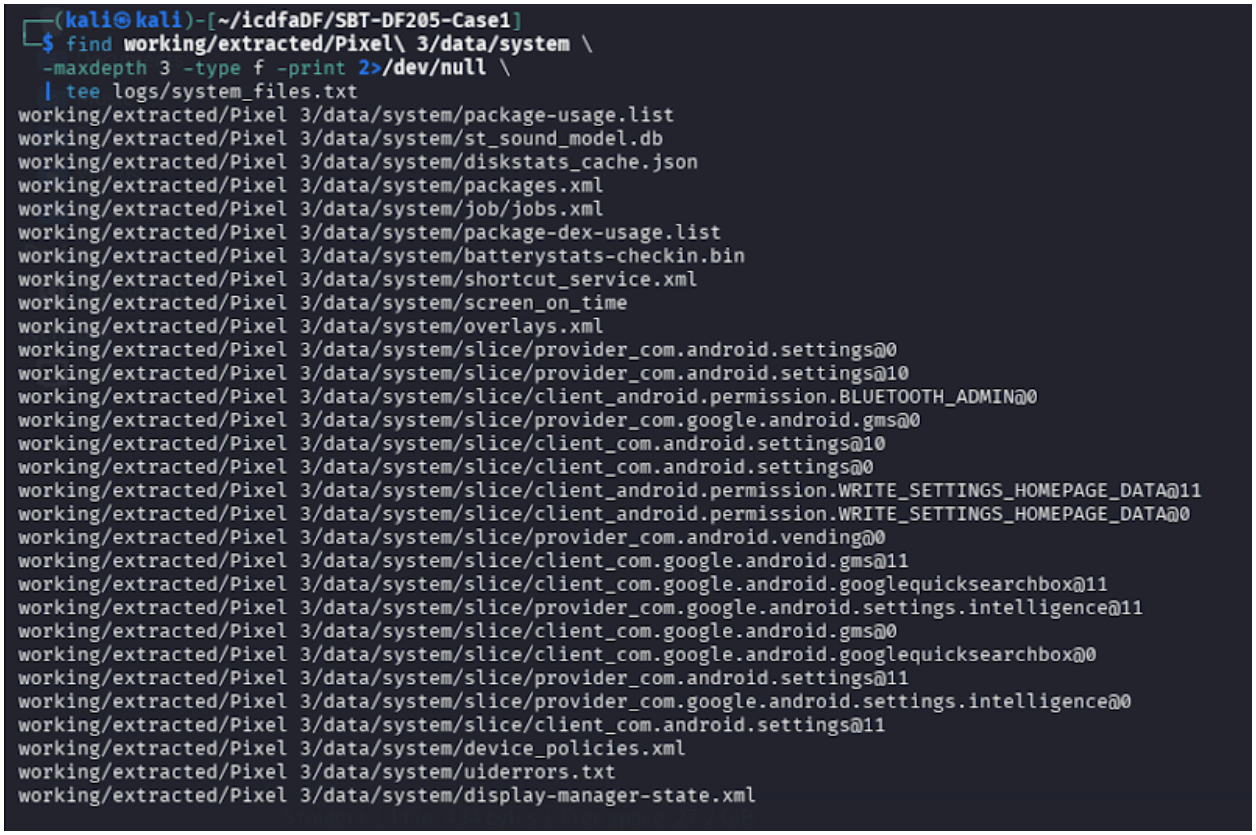


Android account-manager records

find working/extracted/Pixel\ 3/data/system \

  -maxdepth 3 -type f -print 2>/dev/null \

  | tee logs/system_files.txt



Inspect Android user profiles

cd ~/icdfaDF/SBT-DF205-Case1

for f in \

  "working/extracted/Pixel 3/data/system/users/userlist.xml" \

```
  "working/extracted/Pixel 3/data/system/users/0.xml" \
  "working/extracted/Pixel 3/data/system/users/11.xml"
do
    echo "========================================"
    echo "SOURCE: $f"
    if [ -f "$f" ]; then
        sed -n '1,160p' "$f"
    else
        echo "File not found"
    fi
done | tee logs/user_profile_details.txt
```



Identify account tables

Command

```
cd ~/icdfaDF/SBT-DF205-Case1

for db in \
  "working/extracted/Pixel 3/data/system_ce/0/accounts_ce.db" \
  "working/extracted/Pixel 3/data/system_de/0/accounts_de.db" \
  "working/extracted/Pixel 3/data/system_de/11/accounts_de.db"
do
    echo "========================================="
    echo "DATABASE: $db"
    if [ -f "$db" ]; then
        sqlite3 "$db" ".tables"
        sqlite3 "$db" ".schema"
    else
        echo "Database not found"
    fi
```

done | tee logs/account_database_schema.txt



Extract account names and types

Run this command to query only _id, name, and type from each profile’s account database.

cd ~/icdfaDF/SBT-DF205-Case1

for db in \

  "working/extracted/Pixel 3/data/system_ce/0/accounts_ce.db" \

  "working/extracted/Pixel 3/data/system_de/0/accounts_de.db" \

  "working/extracted/Pixel 3/data/system_de/11/accounts_de.db"

do

  echo "=="

```
    echo "DATABASE: $db"
    if [ -f "$db" ]; then
        sqlite3 -header -column "$db" \
          "SELECT _id, name, type FROM accounts ORDER BY _id;"
    fi
done | tee logs/account_names_types.txt
```

```
do
    echo "═══════════════════════════════════════"
    echo "DATABASE: $db"
    if [ -f "$db" ]; then
        sqlite3 -header -column "$db" \
        "SELECT _id, name, type FROM accounts ORDER BY _id;"
    fi
done | tee logs/account_names_types.txt
═══════════════════════════════════════
DATABASE: working/extracted/Pixel 3/data/system_ce/0/accounts_ce.db
_id         name                  type
──  ──────────────────  ──────────────────────────────
  1  thisisdfir@gmail.com  com.google
  2  858233690             org.telegram.messenger
  3  Signal                org.thoughtcrime.securesms
  4  imo HD                com.imo.android.imous
  5  TikTok                com.zhiliaoapp.musically
  6  Messenger             com.facebook.messenger
  7  TDfir                 com.twitter.android.auth.login
  8  WhatsApp              com.whatsapp
 10  thisisdfir            com.silentcircle.account
 11  Skype                 com.skype.raider
 12  TextNow               com.enflick.android.TextNow.account
 13  +19195794674          com.viber.voip
 14  Duo                   com.google.android.apps.tachyon
═══════════════════════════════════════
DATABASE: working/extracted/Pixel 3/data/system_de/0/accounts_de.db
_id         name                  type
──  ──────────────────  ──────────────────────────────
  1  thisisdfir@gmail.com  com.google
  2  858233690             org.telegram.messenger
  3  Signal                org.thoughtcrime.securesms
  4  imo HD                com.imo.android.imous
  5  TikTok                com.zhiliaoapp.musically
  6  Messenger             com.facebook.messenger
  7  TDfir                 com.twitter.android.auth.login
  8  WhatsApp              com.whatsapp
 10  thisisdfir            com.silentcircle.account
 11  Skype                 com.skype.raider
 12  TextNow               com.enflick.android.TextNow.account
 13  +19195794674          com.viber.voip
 14  Duo                   com.google.android.apps.tachyon
```

Settings and connectivity

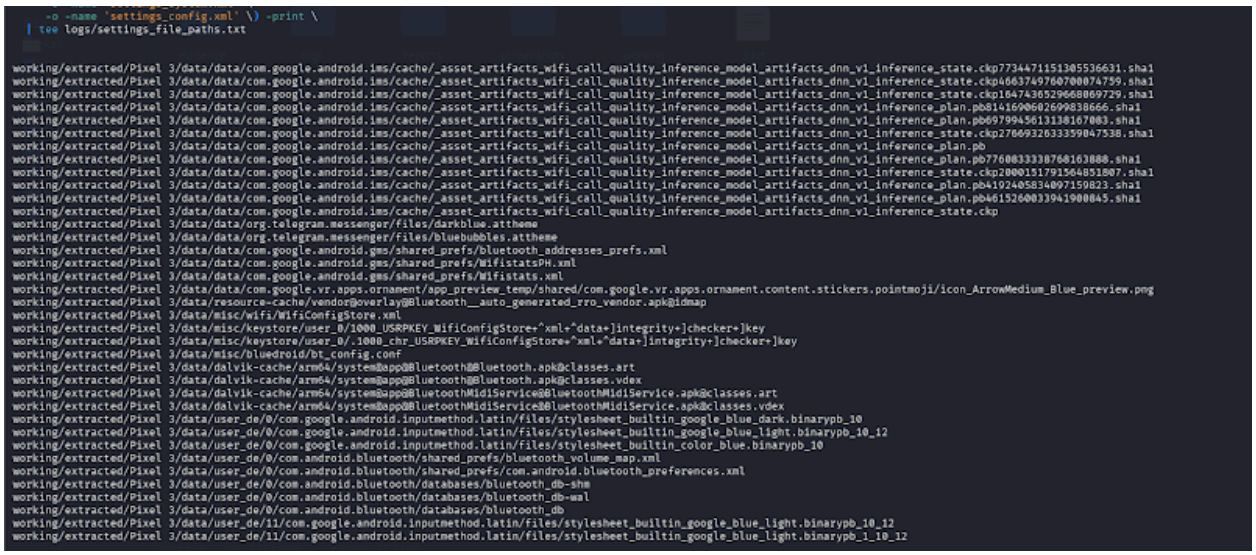
First, locate the Wi-Fi and Bluetooth configuration files and record the settings files available for review. Run:

cd ~/icdfaDF/SBT-DF205-Case1

find "working/extracted/Pixel 3/data" -type f \

```
  \( -iname '*wifi*' \
     -o -iname '*bluetooth*' \
     -o -iname 'bt_config.conf' \
     -o -iname '*blue*' \) -print \
  | tee logs/connectivity_file_paths.txt

find "working/extracted/Pixel 3/data/system/users/0" \
  -maxdepth 1 -type f \
  \( -name 'settings_global.xml' \
     -o -name 'settings_secure.xml' \
     -o -name 'settings_system.xml' \
     -o -name 'settings_config.xml' \) -print \
  | tee logs/settings_file_paths.txt
```



Settings and connectivity

The Wi-Fi configuration file data/misc/wifi/WifiConfigStore.xml contains a MAC-address mapping associated with the configuration key "CcookiesDcastleR5"WPA_PSK and address ae:f6:c7:a8:a5:c2. The Wi-Fi configuration files also contain encrypted data. The examined output does not establish when the associated network was connected to, or whether a connection was successful.

The Bluetooth configuration file data/misc/bluedroid/bt_config.conf records the address 7c:d9:5c:ac:a2:ce and device name Rouge. The Bluetooth preferences file data/user_de/0/com.android.bluetooth/shared_prefs/com.android.bluetooth_preferences.xml records one contact and a lastUpdatedTimestamp value of 1581273845786 milliseconds.

These artefacts establish that Wi-Fi configuration and Bluetooth metadata are present in the extraction. They do not, by themselves, prove a particular network session, physical proximity, or a successful Bluetooth connection.

command

cd ~/icdfaDF/SBT-DF205-Case1

grep -hE '<setting ' \

```
  "working/extracted/Pixel 3/data/system/users/0/settings_global.xml" \

  "working/extracted/Pixel 3/data/system/users/0/settings_secure.xml" \

  "working/extracted/Pixel 3/data/system/users/0/settings_system.xml" \

  "working/extracted/Pixel 3/data/system/users/0/settings_config.xml" \

  > logs/settings_entries.txt

grep -Ein
'SSID|BSSID|ConfigKey|HiddenSSID|CreatorUid|LastUpdateUid|Bluetooth|address|name' \

  "working/extracted/Pixel 3/data/misc_ce/0/wifi/WifiConfigStore.xml" \

  "working/extracted/Pixel 3/data/misc/wifi/WifiConfigStore.xml" \

  "working/extracted/Pixel 3/data/misc/bluedroid/bt_config.conf" \

  "working/extracted/Pixel
3/data/user_de/0/com.android.bluetooth/shared_prefs/com.android.bluetooth_preferences.xml" \

  2>/dev/null \

  | sed -E 's/(PreSharedKey|Password|KeyMaterial|WEPKeys)[^,>]*/\1=[REDACTED]/Ig' \

  > logs/connectivity_relevant_fields.txt

cat logs/connectivity_relevant_fields.txt
```



SMS/MMS, contacts, call logs, and calendar artefacts

Findings

SMS: 24 records across 14 threads in the primary SMS database. Thread 13 contains eight Facebook-related notifications; thread 17 contains two outgoing SMS messages.

MMS: Seven records associated with thread 17. Message parts contain text discussing Signal and messaging through the Messages app.

Contacts: Multiple application-specific contact entries link to contact ID 10 and include the phone number 9195790479 in several formats. These entries establish a database association, not a confirmed real-world identity.

Call log: The three inspected records are dated 31 January 2020 UTC. All have blank number fields, zero duration, and call type value 3. They do not establish a caller or recipient from the fields inspected.

Calendar: The Events, Attendees, Instances, and Reminders tables each contain zero records.

Identify relevant databases

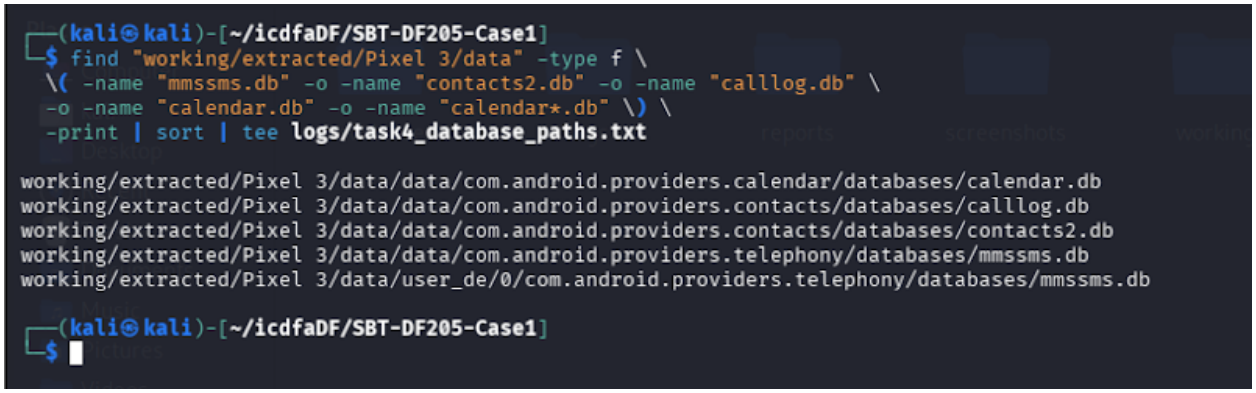
Command used:

```
find "working/extracted/Pixel 3/data" -type f \

  \( -name "mmssms.db" -o -name "contacts2.db" -o -name "calllog.db" \

  -o -name "calendar.db" -o -name "calendar*.db" \) \

  -print | sort | tee logs/task4_database_paths.txt
```



Verify database integrity and record metadata

Command used:

```
sha256sum "working/extracted/Pixel
3/data/data/com.android.providers.calendar/databases/calendar.db" \

  "working/extracted/Pixel 3/data/data/com.android.providers.contacts/databases/calllog.db" \

  "working/extracted/Pixel 3/data/data/com.android.providers.contacts/databases/contacts2.db" \

  "working/extracted/Pixel 3/data/data/com.android.providers.telephony/databases/mmssms.db"
\
```

"working/extracted/Pixel 3/data/user_de/0/com.android.providers.telephony/databases/mmssms.db" \

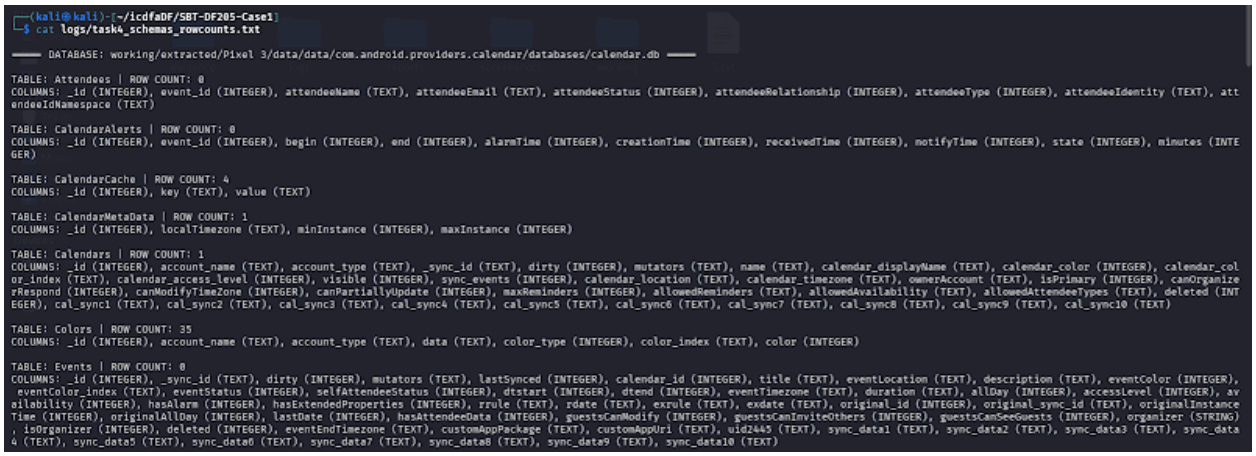
| tee logs/task4_database_hashes.txt



Inspect database schemas and record counts

Command used

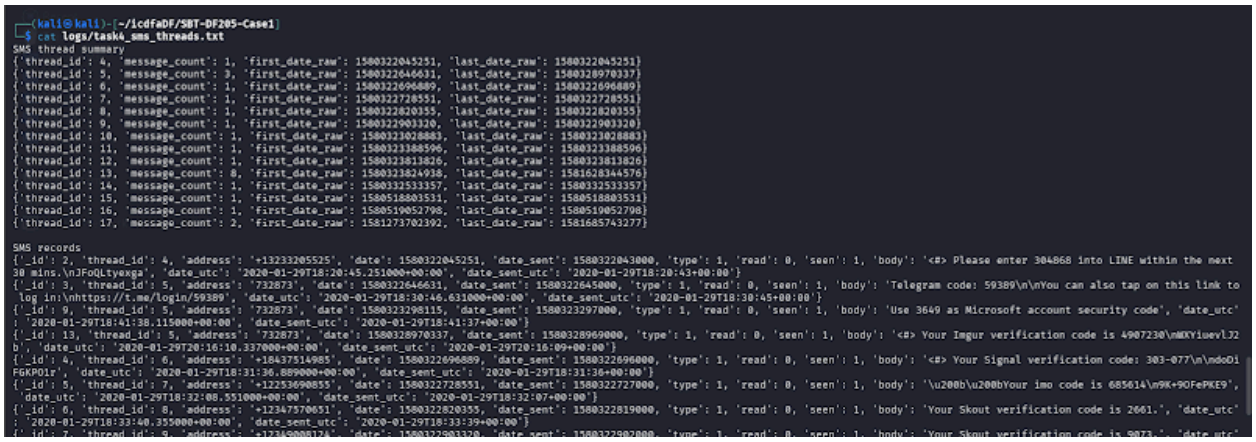
logs/task4_schemas_rowcounts.txt



Extract SMS message records

Command

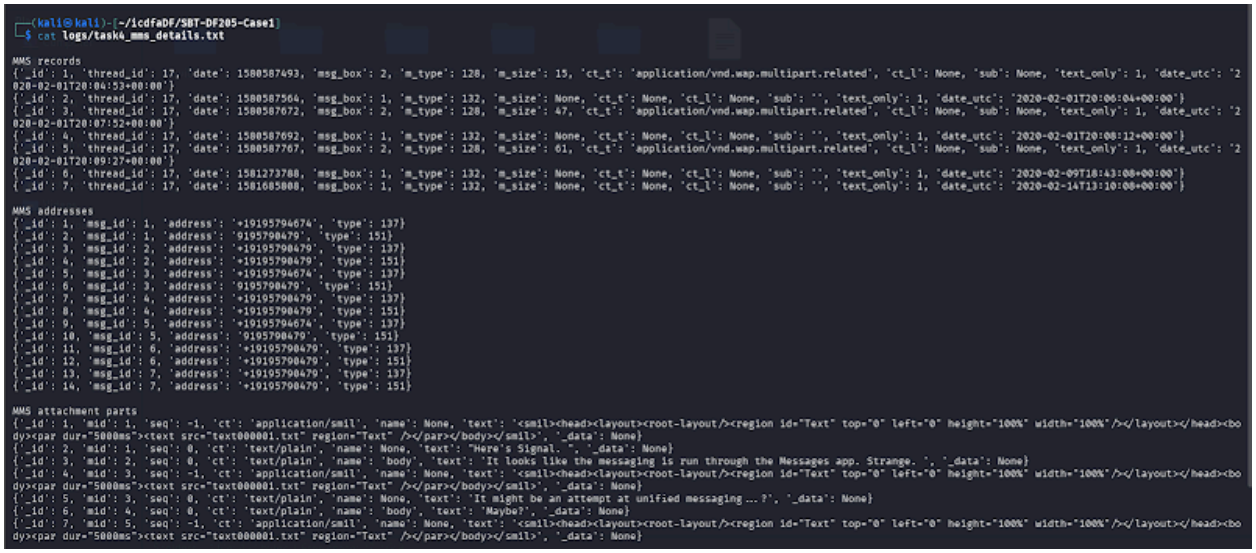
logs/task4_sms_threads.txt



Extract MMS messages, addresses and parts

Command

logs/task4_mms_details.txt



Google Messages and Telephony Provider

Three SMS records were matched using record ID, sender address, message content, and timestamps.

SMS ID	Address	UTC timestamp	Finding
2	+13233205525	2020-01-29 18:20:45	Content and address matched
3	732873	2020-01-29 18:30:46	Content and address matched
4	+18437514985	2020-01-29 18:31:36	Content and address matched

The Google Messages sms_message_uri values (content://sms/2, /3, and /4) correspond to the Telephony Provider record IDs. The message timestamps also align: the provider's date matches the Google Messages received_timestamp, and date_sent aligns with sent_timestamp for these records.

B. Google Dialer and AOSP call log

The SQL comparison confirmed four matching calls.

Record ID	UTC timestamp	Duration	Call type
11	2020-02-14 01:29:01	11 seconds	2
12	2020-02-14 01:30:24	65 seconds	1
13	2020-02-14 13:05:57	86 seconds	2
14	2020-02-14 14:41:18	91 seconds	1

Each match has the same timestamp, duration, and call-type value in both databases. Phone-number formatting differs, but the normalized numbers are consistent.

The AOSP call log also contains two additional 3-second calls, on 12 February and 13 February 2020, with type value 4. These were not present in the supplied annotated call-log output.

C. File-transfer table

The transfers table in imsft.db contains zero records. This finding applies only to that table and does not establish that no transfers occurred through other applications or mechanisms.

D. Timestamp discrepancy

Two outgoing Google Messages records have a sent_timestamp that renders as 1970-01-01 00:00:00, while their received_timestamp values are in February 2020. This discrepancy should be documented; the epoch-like timestamp should not be treated as the actual sending time without further verification.

Locate Google Messages database and sidecars

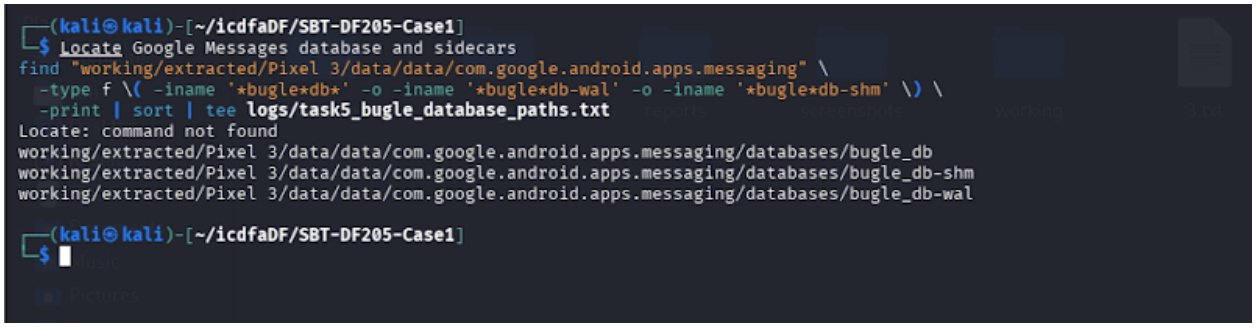
find "working/extracted/Pixel 3/data/data/com.google.android.apps.messaging" \

-type f

$-iname' * bugle * db * ' -o -iname' * bugle * db -wal' -o -iname' * bugle * db -shm'$

\

-print | sort | tee logs/task5_bugle_database_paths.txt



List Google Messages tables

sqlite3 -readonly \

"working/extracted/Pixel 3/data/data/com.google.android.apps.messaging/databases/bugle_db" \

"SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;" \

| tee logs/task5_bugle_tables.txt



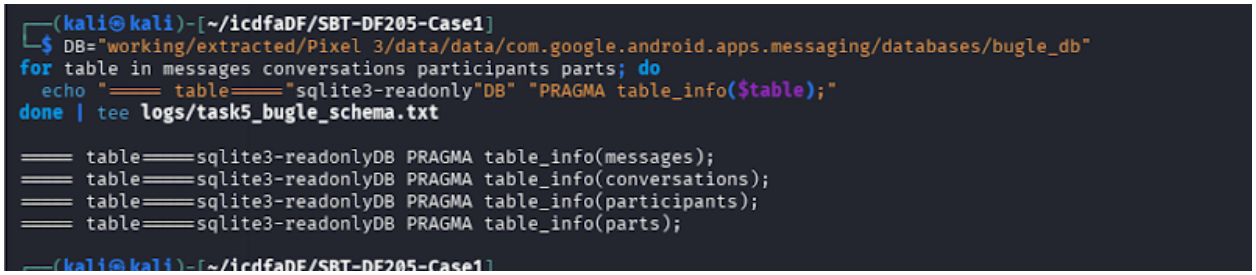
Inspect relevant table schemas

DB="working/extracted/Pixel 3/data/data/com.google.android.apps.messaging/databases/bugle_db"

for table in messages conversations participants parts; do

echo "===== $table ===== "sqlite3 -readonly" DB" "PRAGMA table_info($table);"

done | tee logs/task5_bugle_schema.txt



Extract Google Messages records

sqlite3 -header -column -readonly \

"working/extracted/Pixel 3/data/data/com.google.android.apps.messaging/databases/bugle_db" "

SELECT

m._id AS message_id,

m.conversation_id,

datetime(m.sent_timestamp/1000,'unixepoch') AS sent_utc,

datetime(m.received_timestamp/1000,'unixepoch') AS received_utc,

m.message_protocol,

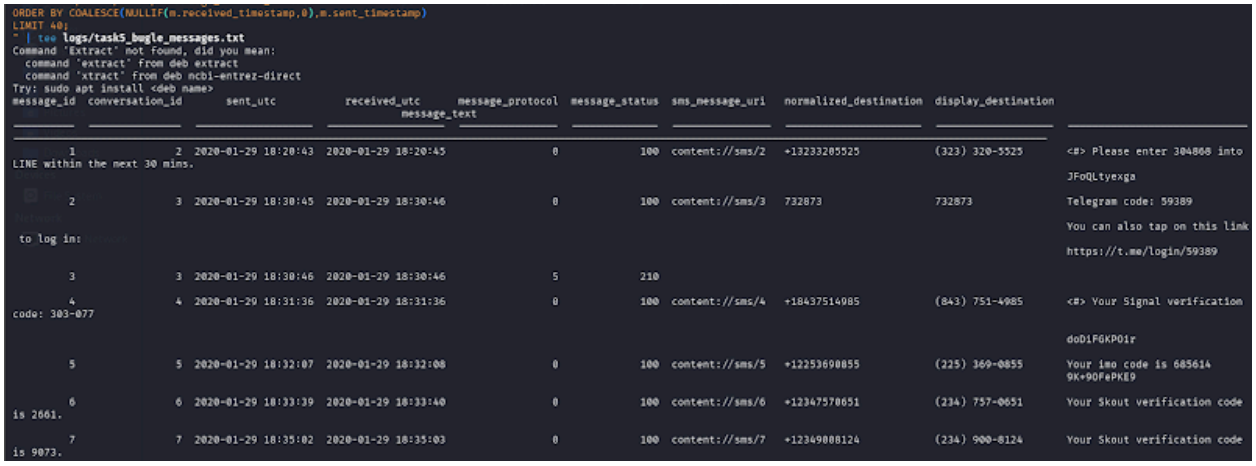
m.message_status,

m.sms_message_uri,

p.normalized_destination,

p.display_destination,

```
  pt.text AS message_text
FROM messages m
LEFT JOIN participants p ON p._id = m.sender_id
LEFT JOIN parts pt ON pt.message_id = m._id
ORDER BY COALESCE(NULLIF(m.received_timestamp,0),m.sent_timestamp)
LIMIT 40;
" | tee logs/task5_bugle_messages.txt
```



```
ORDER BY COALESCE(NULLIF(m.received_timestamp,0),m.sent_timestamp)
LIMIT 40;
" | tee logs/task5_bugle_messages.txt
Command 'Extract' not found, did you mean:
  command 'extract' from deb extract
  command 'xtract' from deb ncbi-entrez-direct
Try: sudo apt install <deb name>
message_id  conversation_id       sent_utc          received_utc      message_protocol  message_status  sms_message_uri   normalized_destination  display_destination
                                                          message_text
──────────  ──────────────  ──────────────────  ──────────────────  ────────────────  ──────────────  ────────────────  ──────────────────────  ───────────────────
──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
          1               2  2020-01-29 18:20:43  2020-01-29 18:20:45                 0             100  content://sms/2   +13233205525            (323) 320-5525      <#> Please enter 304868 into
LINE within the next 30 mins.
                                                                                                                                                                  JFoQLtyexga

          2               3  2020-01-29 18:30:45  2020-01-29 18:30:46                 0             100  content://sms/3   732873                  732873              Telegram code: 59389
                                                                                                                                                                  You can also tap on this link
 to log in:
                                                                                                                                                                  https://t.me/login/59389

          3               3  2020-01-29 18:30:46  2020-01-29 18:30:46                 5             210

          4               4  2020-01-29 18:31:36  2020-01-29 18:31:36                 0             100  content://sms/4   +18437514985            (843) 751-4985      <#> Your Signal verification
code: 303-077
                                                                                                                                                                  doDiFGKPOir

          5               5  2020-01-29 18:32:07  2020-01-29 18:32:08                 0             100  content://sms/5   +12253690855            (225) 369-0855      Your imo code is 685614
                                                                                                                                                                  9K+9OFePKE9

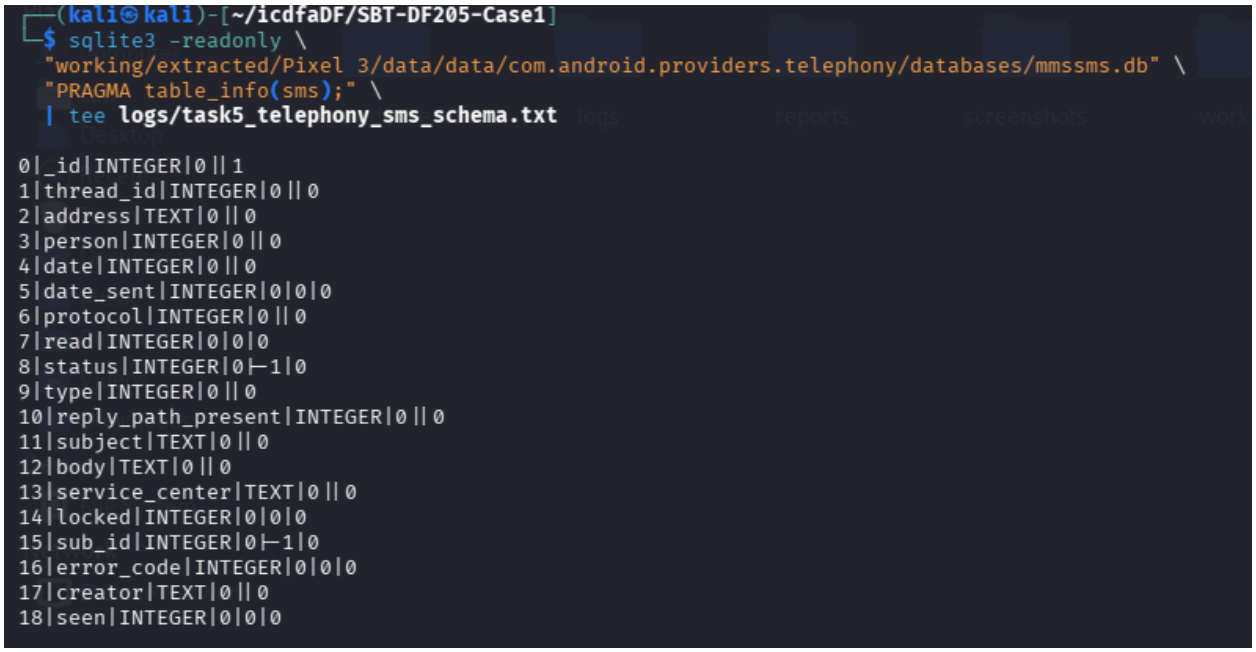
          6               6  2020-01-29 18:33:39  2020-01-29 18:33:40                 0             100  content://sms/6   +12347570651            (234) 757-0651      Your Skout verification code
is 2661.

          7               7  2020-01-29 18:35:02  2020-01-29 18:35:03                 0             100  content://sms/7   +12349008124            (234) 900-8124      Your Skout verification code
is 9073.
```

Inspect the Telephony Provider SMS schema

```
sqlite3 -readonly \
  "working/extracted/Pixel 3/data/data/com.android.providers.telephony/databases/mmssms.db" \
  "PRAGMA table_info(sms);" \
```

| tee logs/task5_telephony_sms_schema.txt



Verify three linked SMS records

sqlite3 -header -column -readonly \

"working/extracted/Pixel 3/data/data/com.android.providers.telephony/databases/mmssms.db"
"

SELECT

_id AS sms_id,

address,

datetime(date/1000,'unixepoch') AS date_utc,

datetime(date_sent/1000,'unixepoch') AS date_sent_utc,

```
  type,
  substr(body,1,45) AS body_preview
FROM sms
WHERE _id IN (2,3,4)
ORDER BY _id;
" | tee logs/task5_telephony_sms_matches.txt
```

Inspect the AOSP call-log schema

```
sqlite3 -readonly \
  "working/extracted/Pixel 3/data/data/com.android.providers.contacts/databases/calllog.db" \
  "PRAGMA table_info(calls);" \
  | tee logs/task5_aosp_calllog_schema.txt
```



Query AOSP calls for relevant numbers

```
sqlite3 -header -column -readonly \
  "working/extracted/Pixel 3/data/data/com.android.providers.contacts/databases/calllog.db" "
SELECT
  _id,
  number,
  datetime(date/1000,'unixepoch') AS call_time_utc,
```

```
  duration,
  type,
  normalized_number,
  formatted_number
FROM calls
WHERE number IN (
  '+332629243087',
  '+19195121037',
  '+15755863247',
  '9195790479',
  '+19195790479',
  '+19195794674'
)
ORDER BY date;
" | tee logs/task5_aosp_calllog_comparison.txt
```

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ sqlite3 -header -column -readonly \
  "working/extracted/Pixel 3/data/data/com.android.providers.contacts/databases/calllog.db" "
SELECT
  _id,
  number,
  datetime(date/1000,'unixepoch') AS call_time_utc,
  duration,
  type,
  normalized_number,
  formatted_number
FROM calls
WHERE number IN (
  '+332629243087',
  '+19195121037',
  '+15755863247',
  '9195790479',
  '+19195790479',
  '+19195794674'
)
ORDER BY date;
" | tee logs/task5_aosp_calllog_comparison.txt

_id     number         call_time_utc      duration  type  normalized_number  formatted_number
---  -------------  -------------------  --------  ----  -----------------  ----------------
  6  +332629243087  2020-02-08 20:14:56         0     3  +332629243087      +332629243087
  7  +19195121037   2020-02-12 15:37:29         0     3  +19195121037       +1 919-512-1037
  8  +19195121037   2020-02-12 15:38:21         3     4  +19195121037       +1 919-512-1037
  9  +15755863247   2020-02-13 00:52:44         0     3  +15755863247       +1 575-586-3247
 10  +15755863247   2020-02-13 00:53:33         3     4  +15755863247       +1 575-586-3247
 11  9195790479     2020-02-14 01:29:01        11     2  +19195790479
 12  +19195790479   2020-02-14 01:30:24        65     1  +19195790479
 13  9195790479     2020-02-14 13:05:57        86     2  +19195790479       (919) 579-0479
 14  9195790479     2020-02-14 14:41:18        91     1  +19195790479       (919) 579-0479
```

Reproduce the call comparison using SQL

sqlite3 -header -column -readonly \

"working/extracted/Pixel 3/data/data/com.google.android.dialer/databases/annotated_call_log.db" "

ATTACH DATABASE 'working/extracted/Pixel 3/data/data/com.android.providers.contacts/databases/calllog.db' AS aosp;

SELECT

a._id AS dialer_id,

datetime(a.timestamp/1000,'unixepoch') AS dialer_time_utc,

```
  a.formatted_number AS dialer_number,
  a.duration AS dialer_duration,
  a.call_type AS dialer_type,
  c._id AS aosp_id,
  c.number AS aosp_number,
  c.duration AS aosp_duration,
  c.type AS aosp_type
FROM AnnotatedCallLog a
JOIN aosp.calls c ON c.date = a.timestamp
WHERE a.duration > 0
ORDER BY a.timestamp;
" | tee logs/task5_call_comparison_sql.txt
```

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ Reproduce the call comparison using SQL
sqlite3 -header -column -readonly \
  "working/extracted/Pixel 3/data/data/com.google.android.dialer/databases/annotated_call_log.db" "
ATTACH DATABASE 'working/extracted/Pixel 3/data/data/com.android.providers.contacts/databases/calllog.db' AS aosp;

SELECT
  a._id AS dialer_id,
  datetime(a.timestamp/1000,'unixepoch') AS dialer_time_utc,
  a.formatted_number AS dialer_number,
  a.duration AS dialer_duration,
  a.call_type AS dialer_type,
  c._id AS aosp_id,
  c.number AS aosp_number,
  c.duration AS aosp_duration,
  c.type AS aosp_type
FROM AnnotatedCallLog a
JOIN aosp.calls c ON c.date = a.timestamp
WHERE a.duration > 0
ORDER BY a.timestamp;
" | tee logs/task5_call_comparison_sql.txt
Reproduce: command not found
dialer_id  dialer_time_utc     dialer_number   dialer_duration  dialer_type  aosp_id  aosp_number    aosp_duration  aosp_type
─────────  ──────────────────  ──────────────  ───────────────  ───────────  ───────  ─────────────  ─────────────  ─────────
11         2020-02-14 01:29:01  (919) 579-0479  11               2            11       9195790479     11             2
12         2020-02-14 01:30:24  +1 919-579-0479  65               1            12       +19195790479   65             1
13         2020-02-14 13:05:57  (919) 579-0479  86               2            13       9195790479     86             2
14         2020-02-14 14:41:18  (919) 579-0479  91               1            14       9195790479     91             1
```

Count file-transfer records

sqlite3 \

  "working/extracted/Pixel 3/data/data/com.google.android.apps.messaging/databases/imsft.db" \

  "SELECT COUNT(*) AS transfer_records FROM transfers;" \

  | tee logs/task5_transfer_count.txt

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ sqlite3 \
  "working/extracted/Pixel 3/data/data/com.google.android.apps.messaging/databases/imsft.db" \
  "SELECT COUNT(*) AS transfer_records FROM transfers;" \
  | tee logs/task5_transfer_count.txt

0

┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
```

WhatsApp and another application

Database and account artefacts

The principal WhatsApp database examined was:

working/extracted/Pixel 3/data/data/com.whatsapp/databases/msgstore.db

The contact database was:

working/extracted/Pixel 3/data/data/com.whatsapp/databases/wa.db

The message database contained several relevant tables, including messages, chat, jid, and message_media. The messages table provided the records used to reconstruct the conversation.

The WhatsApp registration preferences contained country code 1 and the number 9195794674, formatted as (919) 579-4674. The stored verification state was 0. These preferences show that the number was entered in the registration artefacts; they do not independently prove that registration was completed successfully.

4.2 Contact identification

The WhatsApp contacts database contained the following record:

Display name: Josh Hickman

Stored number: 9195790479

JID: 19195790479@s.whatsapp.net

This establishes that the extracted contact database associates the displayed name with that number and JID. It does not independently establish the real-world identity of the account holder.

4.3 Reconstructed conversation

The message records associated with 19195790479@s.whatsapp.net included the following:

Date and time (UTC)	Direction	Message or artefact

8 February 2020, 20:55:14	Outgoing	“Hi there! I switched over.”
8 February 2020, 20:55:14	Outgoing	Additional record with no visible text; media type recorded as 0
8 February 2020, 20:56:05	Incoming	“Awesome!”
8 February 2020, 20:57:05	Incoming	JPEG image-related record
8 February 2020, 20:57:49	Outgoing	“The Osbourne!”
8 February 2020, 20:58:12	Outgoing	JPEG image-related record
9 February 2020, 18:00:14	Outgoing	Media-related record with caption “Here I am!!”
9 February 2020, 18:00:33	Outgoing	Media-related record with caption “Here I am!!”
9 February 2020, 18:05:02	Outgoing	Additional media-related record

The records show an exchange of messages and media-related activity over 8–9 February 2020. The text “Hi there! I switched over.” is followed by the incoming reply “Awesome!” approximately 51 seconds later.

The WhatsApp media join identified an incoming JPEG associated with message ID 14 at 20:57:05 UTC, recorded as 21,890 bytes, and an outgoing JPEG associated with message ID

16 at 20:58:12 UTC, recorded as 87,909 bytes. Their database metadata included media paths and URL fields. These records establish that the database references the media; the message database output alone does not establish the visual contents of those images.

5. Kik Findings

5.1 Account and contact artefacts

The principal Kik database examined was:

working/extracted/Pixel 3/data/data/kik.android/databases/d2c5d8db-28cf-4fa4-81bc-59e5be82eda0.kikDatabase.db

The database integrity check returned ok. The messagesTable contained six message records, and KIKcontactsTable contained 70 contact records.

Kik preferences contained these account-associated values:

Username: ThisIsDFIR

Email: thisisdfir@gmail.com

Email-confirmation flag: false

Verification flag: false

These values are stored preference artefacts. The false confirmation and verification flags should not be interpreted as proof of account ownership or successful verification.

The Kik contact referenced by all six message records was:

Display name: Josh Hickman

Username: joshuahickman1

JID: joshuahickman1_ulc@talk.kik.com

The contact record also showed is_group = 0 and in_roster = 1, indicating that the stored record is marked as a non-group contact in the roster.

5.2 Reconstructed conversation

The Kik database contained six records dated 1 February 2020.

Time (UTC)	Direction	Message content
19:40:13	Incoming	“I don't even know why we are doing Kik. Isn't it shutting down soon?”
19:42:36	Outgoing	“No. It was bought by Whisper, so it is staying online.”
19:43:27	Incoming	“Great (sarcasm).”
19:46:25	Outgoing	“It looks like the video calling feature is gone. And there are ads now.”
19:47:04	Incoming	Image attachment record; no readable body text in the query output
19:48:04	Outgoing	Image attachment record; no readable body text in the query output

The four text records show a discussion about Kik's continued availability, video-calling functionality, and advertisements. Two additional records contain attachment metadata rather than readable text in the queried message body.

5.3 Kik media artefacts

The KIKContentTable and KIKContentURITable records linked the last two messages to image-related content. Both corresponding JPEG files were located in the extracted application's cache directory:

working/extracted/Pixel 3/data/data/kik.android/cache/d2c5d8db-28cf-4fa4-81bc-59e5be82eda0/chatPicsBig/

The files were verified as JPEG images using the file utility.

File	Recorded size	Dimensions	SHA-256
cebcca1c-94a2-4d42-863b-3fd80f3fd233.jpg	90,417 bytes	1600 × 1106	6234af0da89e81a19856250ccbf212e6b2fe60b0d3490fb5fb3ace95897f36e9
06ceeadd-995d-4e3c-be2b-2a0b26dec374.jpg	221,306 bytes	1482 × 1600	5a24f5c9200afe66c7fe809c0edcd7e20cd7644c1d5d263f1ebfcdad9d66e513

The file sizes match the sizes recorded in the Kik content metadata. The matching names, sizes, and content identifiers support the association between the message records and the cached image files. The hashes provide a means of checking file integrity during subsequent analysis. The visual contents of the images should only be described after direct inspection.

6. Cross-Application Correlation

The WhatsApp and Kik contact databases both contain the display name Josh Hickman, but they use different identifiers.

Attribute	WhatsApp	Kik

Stored display name	Josh Hickman	Josh Hickman
Identifier	19195790479@s.whatsapp.net	joshuahickman1_ulc@talk.kik.com
Number or username	9195790479	joshuahickman1

The shared display name supports an association between the contact records across the two applications. However, the databases do not, on the evidence presented here, independently prove that the two identifiers belong to the same real-world person. The correlation should therefore be reported as a supported association rather than a confirmed identity.

7. Database Schema and Message Status

The WhatsApp messages table contains fields including _id, key_remote_jid, key_from_me, status, data, timestamp, media_name, media_mime_type, and media_size.

The Kik messagesTable contains fields including _id, body, partner_jid, was_me, read_state, timestamp, and content_id.

The observed WhatsApp status distribution was:

key_from_me	status	Record count
0	-1	1
0	0	8
0	13	1

1	6	3
1	13	6

All six Kik message records had read_state = 500.

These values were recorded directly from the databases. Their meanings were not independently validated against documentation or source code for the specific application versions represented in the evidence. Consequently, no definitive sent, delivered, or read int**erpretation is assigned to these numeric values.**

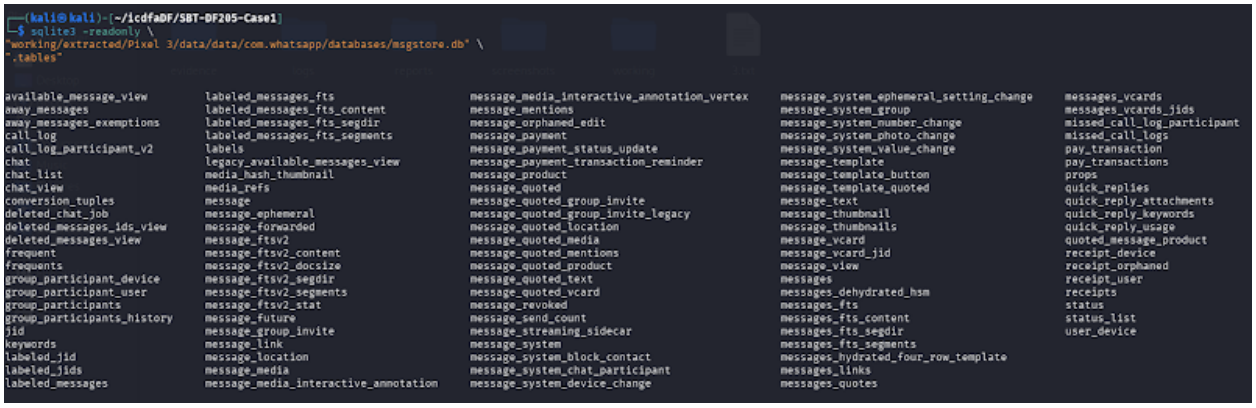
inspect WhatsApp database tables

cd ~/icdfaDF/SBT-DF205-Case1

sqlite3 -readonly \

"working/extracted/Pixel 3/data/data/com.whatsapp/databases/msgstore.db" \

".tables"



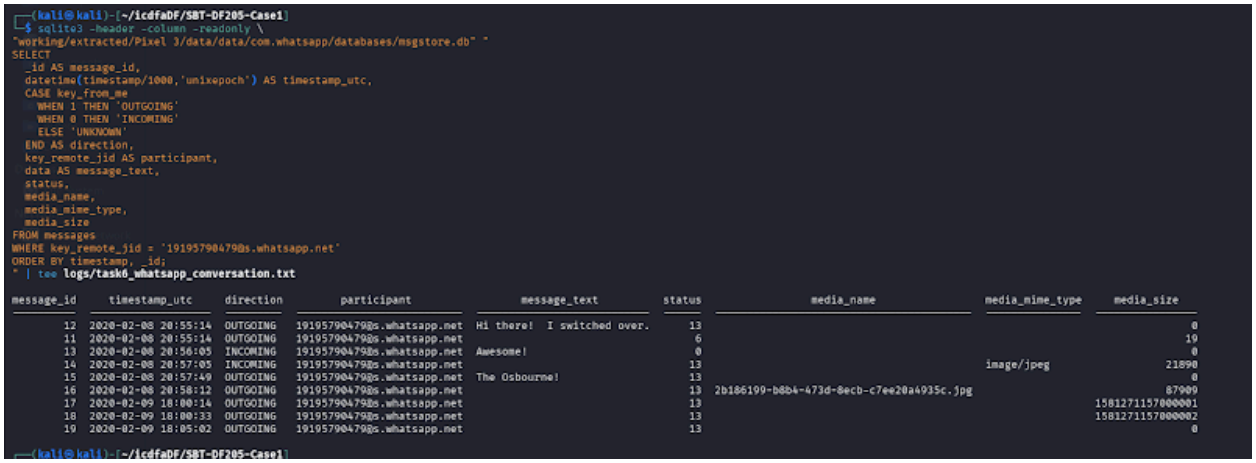
reconstruct WhatsApp messages

```
sqlite3 -header -column -readonly \
"working/extracted/Pixel 3/data/data/com.whatsapp/databases/msgstore.db" "
SELECT
  _id AS message_id,
  datetime(timestamp/1000,'unixepoch') AS timestamp_utc,
  CASE key_from_me
    WHEN 1 THEN 'OUTGOING'
    WHEN 0 THEN 'INCOMING'
    ELSE 'UNKNOWN'
  END AS direction,
  key_remote_jid AS participant,
  data AS message_text,
  status,
  media_name,
  media_mime_type,
  media_size
FROM messages
```

WHERE key_remote_jid = '19195790479@s.whatsapp.net'

ORDER BY timestamp, _id;

" | tee logs/task6_whatsapp_conversation.txt



identify the WhatsApp contact

sqlite3 -header -column -readonly \

"working/extracted/Pixel 3/data/data/com.whatsapp/databases/wa.db" "

SELECT jid, number, display_name, wa_name

FROM wa_contacts

WHERE display_name LIKE '%Josh Hickman%'

  OR wa_name LIKE '%Josh Hickman%';

"



reconstruct Kik messages

KIK="working/extracted/Pixel 3/data/data/kik.android/databases/d2c5d8db-28cf-4fa4-81bc-59e5be82eda0.kikDatabase.db"

sqlite3 -header -column -readonly "$KIK" "

SELECT

m._id AS message_id,

datetime(m.timestamp/1000,'unixepoch') AS timestamp_utc,

CASE m.was_me

WHEN 1 THEN 'OUTGOING'

WHEN 0 THEN 'INCOMING'

ELSE 'UNKNOWN'

END AS direction,

```
  m.partner_jid,

  c.display_name AS contact_name,

  c.user_name AS contact_username,

  m.body,

  m.content_id

FROM messagesTable m

LEFT JOIN KIKcontactsTable c ON c.jid = m.partner_jid

ORDER BY m.timestamp, m._id;

" | tee logs/task6_kik_messages.txt
```



inspect Kik attachment metadata

sqlite3 -header -column -readonly "$KIK" "

```
SELECT

  m._id AS message_id,

  datetime(m.timestamp/1000,'unixepoch') AS timestamp_utc,

  m.content_id,

  c.content_name,

  c.content_string

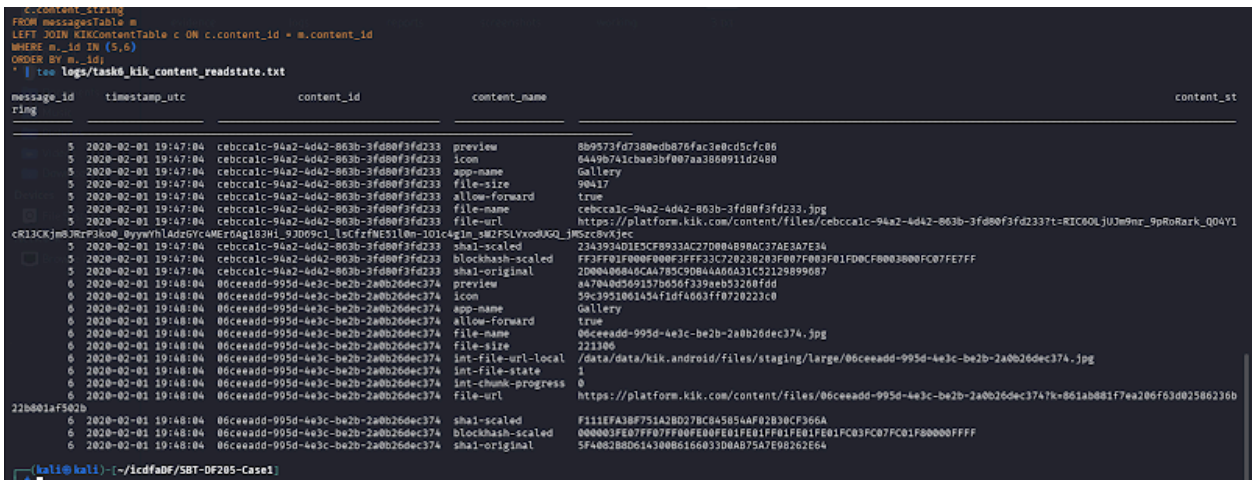
FROM messagesTable m

LEFT JOIN KIKContentTable c ON c.content_id = m.content_id

WHERE m._id IN (5,6)

ORDER BY m._id;

" | tee logs/task6_kik_content_readstate.txt
```



verify the recovered image files

$IMGDIR/cebcca1c-94a2-4d42-863b-3fd80f3fd233.jpg"file"$

IMGDIR="working/extracted/Pixel 3/data/data/kik.android/cache/d2c5d8db-28cf-4fa4-81bc-59e5be82eda0/chatPicsBig"

file "IMGDIR/06ceeadd-995d-4e3c-be2b-2a0b26dec374.jpg"

sha256sum \

"$IMGDIR/cebcca1c-94a2-4d42-863b-3fd80f3fd233.jpg" \

"$IMGDIR/06ceeadd-995d-4e3c-be2b-2a0b26dec374.jpg" \

| tee logs/task6_kik_image_verification.txt



correlate the contact across applications

sqlite3 -header -column -readonly \

"working/extracted/Pixel 3/data/data/com.whatsapp/databases/wa.db" \

"SELECT jid, number, display_name

FROM wa_contacts

WHERE display_name = 'Josh Hickman';"

sqlite3 -header -column -readonly "$KIK" \

"SELECT jid, display_name, user_name

FROM KIKcontactsTable

WHERE jid = 'joshuahickman1_ulc@talk.kik.com';"



document observed status values

sqlite3 -header -column -readonly \

"working/extracted/Pixel 3/data/data/com.whatsapp/databases/msgstore.db" \

"SELECT key_from_me, status, COUNT(*) AS records

FROM messages

GROUP BY key_from_me, status

ORDER BY key_from_me, status;"

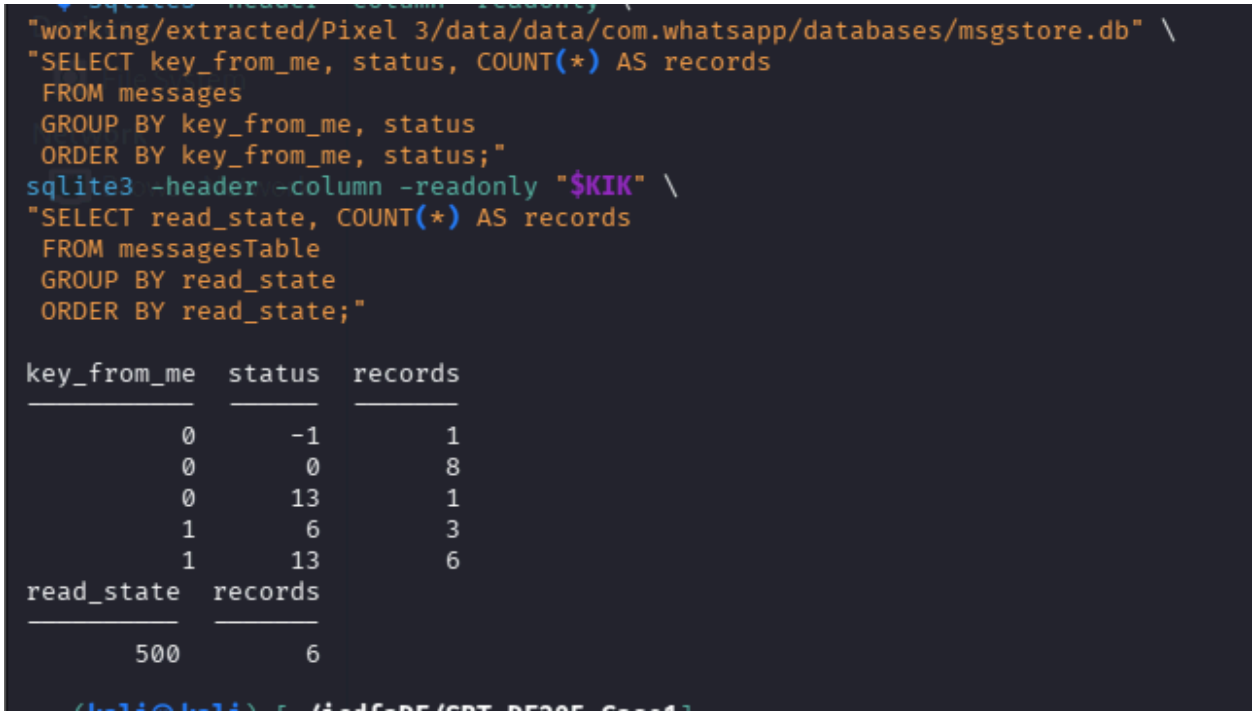
sqlite3 -header -column -readonly "$KIK" \

"SELECT read_state, COUNT(*) AS records

FROM messagesTable

GROUP BY read_state

ORDER BY read_state;"



Maps and photographs
Findings

Google Maps location-related record

The Maps database gmm_myplaces.db contained a record in the sync_item table with the following details:

Field	Observed value
Database	gmm_myplaces.db
Table	sync_item
Corpus	8
Key string	1:0
Record timestamp	1554407343604 milliseconds
Converted timestamp	2019-04-04 19:49:03 UTC
Latitude	35.734602
Longitude	-78.636654
Address text	121 E Tryon Rd, Raleigh, NC 27603
Maps URL text	http://maps.google.com/?q=121+E+Tryon+Rd,+Raleigh,+NC+27603&ftid=...

is_local	0

The record contains an address and corresponding coordinates. It is therefore a location-related Maps record. The available output does not conclusively establish whether the record represents a search, saved place, or another Maps item. The timestamp is the record's timestamp and should not automatically be interpreted as a visit time.

Second Maps record: A separate second location entry could not be confirmed from the examined records. The userevent3_table in ue3.db returned no rows, and the two tables examined in ugc_photos_location_data.db also returned zero rows. The uri record in gmm_storage.db contained Maps-related serialized data, but the inspected excerpt did not establish a distinct second location. This limitation is reported rather than treating an unverified record as a confirmed location.

4.2 Photographic evidence

Three records were extracted from the Google Photos local_media table in gphotos0.db.

ID	Filename	Capture timestamp (UTC)	Stored timezone offset	GPS coordinates
3	IMG_20190917_082637.jpg	2019-09-17 12:26:37	-14400000	35.7356, -78.6364
24	IMG_20200209_132353.jpg	2020-02-09 17:03:53	-18000000	35.661, -78.8799
25	IMG_20200213_195030.jpg	2020-02-13 21:10:31	-18000000	35.6589, -78.8742

The timestamps above are conversions of the millisecond values returned in the latest database query. The stored timezone offsets are reproduced as recorded. The database field conventions should be verified before using these values to calculate local capture times.

Photograph 1

Database ID: 3

Filename: IMG_20190917_082637.jpg

Extracted path: working/extracted/Pixel 3/data/media/0/DCIM/Restored/IMG_20190917_082637.jpg

Dimensions: 3024 × 4032 pixels

Size: 5,754,953 bytes

GPS: latitude 35.7356, longitude -78.6364

SHA-256: 8c2a100a6a11aeccbfc196439c6d7d1cc0a149b3dd18264747735b7201bc1177

Photograph 2

Database ID: 24

Filename: IMG_20200209_132353.jpg

Extracted path: working/extracted/Pixel 3/data/media/0/DCIM/Camera/IMG_20200209_132353.jpg

Dimensions: 3024 × 4032 pixels

Size: 3,970,621 bytes

GPS: latitude 35.661, longitude -78.8799

SHA-256:
308df52bdab5aa52e9e82b2a853049c688450c2ff6ad76cd75f0c063e61853fa

Photograph 3

Database ID: 25

Filename: IMG_20200213_195030.jpg

Extracted path: working/extracted/Pixel 3/data/media/0/DCIM/Camera/IMG_20200213_195030.jpg

Dimensions: 3024 × 4032 pixels

Size: 3,450,708 bytes

GPS: latitude 35.6589, longitude -78.8742

SHA-256:
2ed43a0a7b01a58f89c23263542282d3d5713b5cd2f22d7b3d2f56a109145ea6

The hashes identify the exact image-file contents examined and provide a means of checking file integrity in later analysis. They do not independently establish the origin or authenticity of the images.

4.3 Maps and photograph comparison

The Maps record identifies 121 E Tryon Rd, Raleigh, NC 27603, with coordinates 35.734602, -78.636654 and a record timestamp of 4 April 2019.

Photograph 1 has GPS coordinates 35.7356, -78.6364, which are geographically close to the Maps coordinates. However, the photographic record's UTC timestamp is 17 September 2019, several months after the Maps record timestamp.

This provides a geographic similarity between a Maps location-related record and a photograph's stored GPS metadata. The difference in timestamps and the lack of a confirmed

second Maps entry limit the strength of the comparison. The evidence does not establish that the same person created both records, that the Maps timestamp represents a visit, or that the device user physically visited the location at either time.

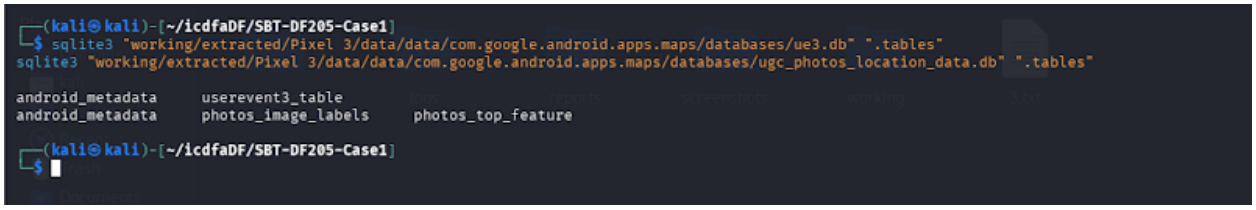
Photographs 2 and 3 contain GPS coordinates in the same general region but at different coordinates. Without additional corroborating records, no stronger conclusion about the events represented by those photographs is justified.

5. Commands Used

Inspect database tables

sqlite3 "working/extracted/Pixel 3/data/data/com.google.android.apps.maps/databases/ue3.db" ".tables"

sqlite3 "working/extracted/Pixel 3/data/data/com.google.android.apps.maps/databases/ugc_photos_location_data.db" ".tables"



Inspect Maps user-event schema and records

sqlite3 -header -column \

"working/extracted/Pixel 3/data/data/com.google.android.apps.maps/databases/ue3.db" \

"PRAGMA table_info(userevent3_table); SELECT * FROM userevent3_table LIMIT 5;" \

| tee logs/task7_maps_userevents.txt



Inspect Maps photo-location tables

sqlite3 -header -column \

"working/extracted/Pixel 3/data/data/com.google.android.apps.maps/databases/ugc_photos_location_data.db" \

"PRAGMA table_info(photos_image_labels); PRAGMA table_info(photos_top_feature); SELECT COUNT(*) AS image_label_rows FROM photos_image_labels; SELECT COUNT(*) AS top_feature_rows FROM photos_top_feature;" \

| tee logs/task7_maps_photo_location_schema.txt

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ sqlite3 -header -column \
"working/extracted/Pixel 3/data/data/com.google.android.apps.maps/databases/ugc_photos_location_data.db" \
"PRAGMA table_info(photos_image_labels); PRAGMA table_info(photos_top_feature); SELECT COUNT(*) AS image_la
ture;" \
| tee logs/task7_maps_photo_location_schema.txt

cid          name           type     notnull  dflt_value  pk
---  ---------------------  -------  -------  ----------  --
  0  photo_uri              TEXT     0                    1
  1  image_label_mid        INTEGER  0                    2
  2  image_label_confidence  REAL    1                    0
cid                 name                      type     notnull  dflt_value  pk
---  ---------------------------------------  -------  -------  ----------  --
  0  photo_uri                                TEXT     0                    1
  1  latitude                                 REAL     0                    0
  2  longitude                                REAL     0                    0
  3  valid_photo_taken_notification_photo     BOOLEAN  1                    0
  4  was_shown_in_photo_taken_notification    BOOLEAN  1                    0
  5  was_uploaded                             BOOLEAN  1                    0
  6  is_face_detected                         BOOLEAN  1                    0
  7  timestamp                                LONG     1                    0
  8  was_dismissed_in_todo_list               BOOLEAN  1                    0
image_label_rows
----------------
              0
top_feature_rows
----------------
              0
```

Extract the three Google Photos records

sqlite3 -header -csv \

"working/extracted/Pixel 3/data/data/com.google.android.apps.photos/databases/gphotos0.db" \

"SELECT _id,filename,filepath,utc_timestamp,timezone_offset,latitude,longitude,width,height,size_bytes FROM local_media WHERE _id IN (3,24,25) ORDER BY _id;" \

| tee logs/task7_three_photo_records.csv

```
┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$ sqlite3 -header -csv \
"working/extracted/Pixel 3/data/data/com.google.android.apps.photos/databases/gphotos0.db" \
"SELECT _id,filename,filepath,utc_timestamp,timezone_offset,latitude,longitude,width,height,size_bytes FROM local_media WHERE _id IN (3,24,25) ORDER BY _id;" \
| tee logs/task7_three_photo_records.csv

_id,filename,filepath,utc_timestamp,timezone_offset,latitude,longitude,width,height,size_bytes
3,IMG_20190917_082637.jpg,/storage/emulated/0/DCIM/Restored/IMG_20190917_082637.jpg,1568723197683,-14400000,35.7356,-78.6364,3024,4032,5754953
24,IMG_20200209_132353.jpg,/storage/emulated/0/DCIM/Camera/IMG_20200209_132353.jpg,1581272633139,-18000000,35.661,-78.8799,3024,4032,3970621
25,IMG_20200213_195030.jpg,/storage/emulated/0/DCIM/Camera/IMG_20200213_195030.jpg,1581641431590,-18000000,35.6589,-78.8742,3024,4032,3450708

┌──(kali㉿kali)-[~/icdfaDF/SBT-DF205-Case1]
└─$
```

Calculate SHA-256 hashes

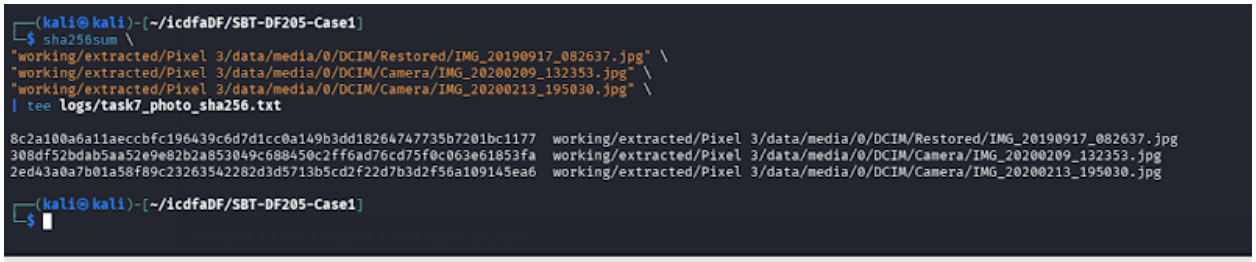
sha256sum \

"working/extracted/Pixel 3/data/media/0/DCIM/Restored/IMG_20190917_082637.jpg" \

"working/extracted/Pixel 3/data/media/0/DCIM/Camera/IMG_20200209_132353.jpg" \

"working/extracted/Pixel 3/data/media/0/DCIM/Camera/IMG_20200213_195030.jpg" \

| tee logs/task7_photo_sha256.txt



Event timeline

The following 14 entries are distinct events drawn from SMS, call logs, MMS, Kik, and WhatsApp. Repeated SMS representations in the Android Messages and Bugle databases are not counted twice.

#	Raw time	Field and format	UTC nversion	vent description	Evidence ID
1	)322045251	; date; Unix ch seconds	)-01-29 0:45.251 ;	; received aining a LINE ication code	4_sms_threads.txt, SMS ID

2	)322646631	S date; Unix ch seconds	)-01-29 0:46.631 ;	S received aining a Telegram ication code	4_sms_threads.txt, SMS ID
3	)322696889	S date; Unix ch seconds	)-01-29 1:36.889 ;	S received aining a Signal ication code	4_sms_threads.txt, SMS ID
4	)323028883	S date; Unix ch seconds	)-01-29 7:08.883 ;	S received aining a WhatsApp ication code	4_sms_threads.txt, SMS ID
5	)483796007	log date; epoch seconds	)-01-31 6:36.007 ;	log record, ID 1; call type 3, duration	4_contact_call_evidence.txt, ID 1
6	)494070836	log date; epoch seconds	)-01-31 7:50.836 ;	log record, ID 2; call type 3, duration	4_contact_call_evidence.txt, ID 2
7	)502115252	log date; epoch seconds	)-01-31 1:55.252 ;	log record, ID 3; call type 3, duration	4_contact_call_evidence.txt, ID 3
8	)587493	S date; Unix ch seconds	)-02-01 4:53 UTC	S record with text e's Signal.”	4_mms_details.txt, MMS ID

9	)-02-01 0:13		stamp_utc; ·style time string	)-02-01 0:13 UTC, xported	ming Kik message ng why the cipants are using	6_kik_messages.txt, sage ID 1
10	)-02-01 2:36		stamp_utc; ·style time string	)-02-01 2:36 UTC, xported	going Kik message ussing the service's inued availability	6_kik_messages.txt, sage ID 2
11	)-02-08 5:14	itsApp	stamp_utc; ·style time string	)-02-08 5:14 UTC, xported	going message: “Hi e! I switched over.”	6_whatsapp_conversation.tx essage ID 12
12	)-02-08 6:05	itsApp	stamp_utc; ·style time string	)-02-08 6:05 UTC, xported	ming reply: esome!”	6_whatsapp_conversation.tx essage ID 13
13	1273702392	s date; Unix ch seconds		)-02-09 1:42.392 ;	sage to/from the ber 9195790479 aining “I am on my to the coffee shop. ou want anything?”	4_sms_threads.txt, SMS ID

14	)-02-14 0:24	log date_utc;)rted UTC time	)-02-14 0:24 UTC, xported	·log record)ciated with +1 ·579-0479; duration econds, raw call 1	5_annotated_call_log.txt, ID 12

Conclusion

The forensic examination of the supplied Google Pixel 3 Android 10 file-system extraction identified communication records across the Android Telephony Provider, Google Messages, Google Dialer, WhatsApp, and Kik. The examination also identified account associations, location-related records in Google Maps, and photographs containing capture-related metadata and GPS coordinates.
The communication records included SMS verification messages associated with several applications, SMS and MMS conversations, call-log entries, WhatsApp messages involving a participant identified in the application records as Josh Hickman, and six Kik messages exchanged with an account displaying the name Josh Hickman. These findings demonstrate recorded activity across multiple communication services. However, verification messages and application registration records do not independently establish successful account registration or prove that a particular individual operated an account.

Cross-source analysis identified the telephone number 9195790479 in contact and communication records and the corresponding WhatsApp participant identifier 19195790479@s.whatsapp.net. This provides a correlation between the contact records and the WhatsApp conversation. Nevertheless, matching telephone numbers and display names do not conclusively establish the real-world identity of an account operator. Overlapping message records in Google Messages and the Android Telephony Provider were treated as duplicate representations of the same communications rather than separate events. The Google Maps examination identified a location-related record containing the address 121 E Tryon Rd, Raleigh, NC 27603, with coordinates 35.734602, -78.636654. Google Photos records also contained GPS coordinates near this location, alongside records associated with other photographed locations. However, the precise meaning of the Maps record was not conclusively established, and the photographic timestamp fields and time-zone offsets require further validation. Accordingly, these records do not independently establish that a particular person visited a specific location at a particular time.
The examined evidence does not establish what confidential company information was allegedly shared, demonstrate that such information was transmitted to an unauthorised recipient, or conclusively attribute the recorded activity to a specific device user. Furthermore, the saved file-transfer output contained no displayed records. This represents a limitation of the available output and does not establish that no file transfer occurred.

Overall assessment: Based on the findings described above, the allegation of unauthorised communication and information sharing remains unresolved. Although the extraction contains evidence of communication activity across several applications, the available records do not sufficiently establish the unauthorised disclosure of company information or identify the individual responsible. Further evidence, including relevant message content, identifiable transferred files, corroborating recipient-side records, and independent user-attribution evidence, would be required to reach a stronger conclusion.